

B 站课程笔记

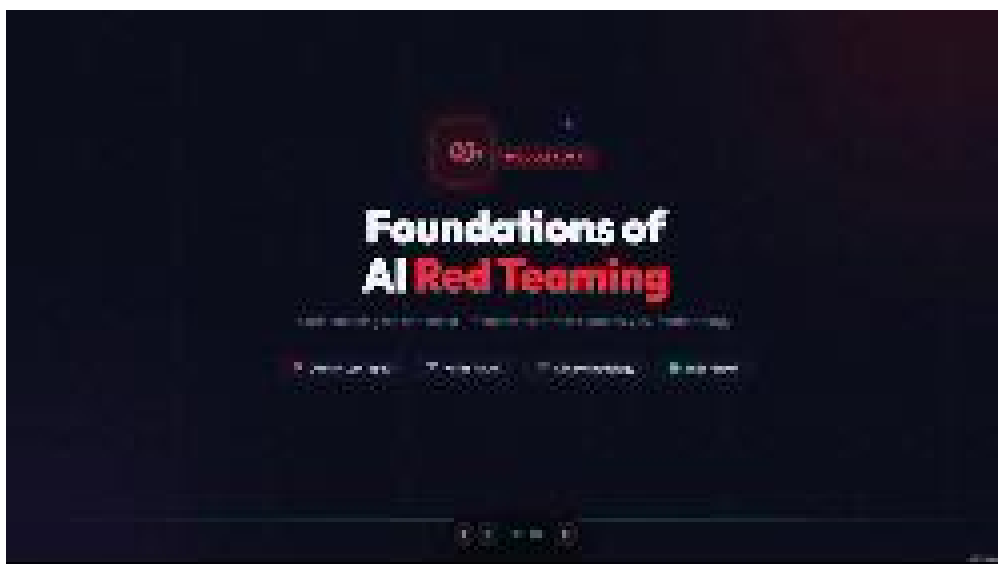
OWASP LLM Top 10 深度解读

AI 红队课：从传统红队到 LLM 应用风险地图

视频作者：洺熙-AI 安全

整理：Codex

生成日期：2026 年 5 月 16 日



视频标题：OWASP LLM Top 10 深度解读

BV 号：BV1AN516BEkk

发布时间：2026-05-13

视频时长：18 分 41 秒

视频链接：<https://www.bilibili.com/video/BV1AN516BEkk>

资料来源：B 站 AI 字幕、视频关键帧、讲者课件画面与 OWASP LLM Top 10 2025 官方术语。

目录

1	从传统红队到 AI 红队	3
1.1	红队的历史谱系	3
1.2	从网络边界迁移到模型边界	3
1.3	目标、工具、产出和确定性的四组对照	5
1.4	AI 红队为什么更像行为压力测试	6
1.5	本章小结	6
2	为什么 AI 需要红队：行为漏洞与不可穷尽输入	7
2.1	通过很多测试后仍可能失败	7
2.2	黑盒不可见性：知道输出，不等于知道内部	8
2.3	无界自然语言输入与组合爆炸	8
2.4	行为漏洞：模型做了不该做的事	9
2.5	RLHF、对齐与安全验证	10
2.6	从测试通过到持续对抗验证	10
2.7	本章小结	11
3	ML 生命周期攻击面地图	11
3.1	从模型结果回看工程流水线	11
3.2	数据收集阶段：投毒从原料开始	11
3.3	训练与微调阶段：后门、Trojan trigger 与 LoRA	12
3.4	评估阶段：覆盖率本身也是攻击面	13
3.5	部署阶段：模型文件、接口和反序列化	13
3.6	推理阶段：用户输入成为实时攻击流	14
3.7	把五阶段连成一条风险传播链	15
3.8	本章小结	15
4	OWASP LLM Top 10 2025 全景图	15
4.1	为什么需要一张标准风险地图	15
4.2	十类风险的官方名称	16
4.3	LLM01:2025 Prompt Injection：提示注入	17

目录	2
4.4 LLM02:2025 Sensitive Information Disclosure: 敏感信息泄露	17
4.5 LLM03:2025 Supply Chain: 供应链	17
4.6 LLM04:2025 Data and Model Poisoning: 数据与模型投毒	18
4.7 LLM05:2025 Improper Output Handling: 输出处理不当	18
4.8 LLM06:2025 Excessive Agency: 过度代理	18
4.9 LLM07:2025 System Prompt Leakage: 系统提示泄露	18
4.10 LLM08:2025 Vector and Embedding Weaknesses: 向量与嵌入弱点	19
4.11 LLM09:2025 Misinformation: 错误信息	19
4.12 LLM10:2025 Unbounded Consumption: 无界消耗	19
4.13 从风险清单到工程分层	19
4.14 本章小结	20
5 关键与高危风险：从指令覆盖到攻击中继	21
5.1 高危风险的共同结构	21
5.2 LLM01:2025 Prompt Injection (提示注入): 控制权争夺	21
5.3 LLM02:2025 Sensitive Information Disclosure (敏感信息泄露): 秘密来自多个层面	22
5.4 LLM03:2025 Supply Chain (供应链): 未经验证的信任雷区	22
5.5 LLM04:2025 Data and Model Poisoning (数据与模型投毒): 把触发器埋进行为	23
5.6 LLM05:2025 Improper Output Handling (输出处理不当): 模型成为攻击中继	23
5.7 沙盒边界与跨系统风险升级	24
5.8 本章小结	25
6 中低与新兴风险：代理、RAG、错误信息与成本失控	25
6.1 中低风险怎样升级	25
6.2 LLM06:2025 Excessive Agency (过度代理): 能力越大，失控面越宽	26
6.3 LLM07:2025 System Prompt Leakage (系统提示泄露): 攻击者拿到操作说明书	27
6.4 LLM08:2025 Vector and Embedding Weaknesses (向量与嵌入弱点): RAG 把知识库变成攻击面	27
6.5 LLM09:2025 Misinformation (错误信息): 幻觉被自动化放大	28
6.6 LLM10:2025 Unbounded Consumption (无界消耗): token、GPU 与递归循环	28
6.7 MITRE ATLAS: 下一讲入口	29

6.8	本章小结	30
6.9	全讲总结与延伸	30
6.9.1	讲者的实质收束	30
6.9.2	核心主张	30
6.9.3	机制压缩	30
6.9.4	实践清单	31
6.9.5	开放问题	31
6.9.6	全讲收束	31
6.9.7	交付件与复核入口	31

1 从传统红队到 AI 红队

1.1 红队的历史谱系

红队早于 AI 时代很久就已经出现。它的源头可以追到冷战时期的军事对抗演练：一组人模拟敌方，主动寻找防御体系的薄弱处；另一组人负责守住系统、流程和人员。这里的核心不只在“攻击”，更关键的是用受控对抗暴露真实防线的错觉。防御者平时看到的是流程、制度和仪表盘；红队要做的是换成敌手视角，问一句更刺耳的问题：这些东西在压力下还站得住吗？

传统红队（Traditional Red Teaming）指在授权范围内模拟真实攻击者，对组织的网络、服务器、Web 应用、身份体系和数据边界进行对抗性测试。它关心的是攻击者能否进入系统、提升权限、横向移动、读取数据，并给出可以复现的证据链。

进入 2000 年代以后，红队方法被网络安全大量吸收。渗透测试、漏洞利用、社会工程、权限维持、日志规避和横向移动，逐渐形成企业安全演练的一套成熟语言。红队报告通常会写清入口、利用步骤、影响范围、修复建议，以及验证修复是否有效的方式。对学习者而言，可以把传统红队想象成“带着敌手剧本做系统体检”：除了看体检单上的正常指标，还要让系统跑步、负重、缺氧，观察它在极端条件下会在哪里失守。

1.2 从网络边界迁移到模型边界

AI 红队继承了这种对抗传统，但对象发生了迁移。传统红队常见目标是网络、服务器、Web 应用、未授权访问和数据窃取；AI 红队的目标扩展到 LLM、机器学习模型、RAG 系统和 Agent。这里需要先把几个基础术语钉牢。

AI 红队（AI Red Teaming）指针对 AI 系统做授权对抗性测试，重点观察模型在恶意输入、异常上下文、检索污染、工具调用和权限压力下会如何行动。它测试的不只是模型能不能回答问题，还包括模型会不会绕过安全过滤器、泄露数据、操纵输出、错误调用工具，或把风险传给下游系统。

LLM（Large Language Model，大语言模型）是以大规模文本、代码或多模态数据训练出的生成模型，典型能力是理解上下文并生成自然语言或代码。LLM 的输入和输出都很像“语言”，这会带来一个安全上的麻烦：语言同时承担内容、指令、上下文和权限暗示，边界天然松动。

RAG（Retrieval-Augmented Generation，检索增强生成）是一种把模型连接到外部知识库的方法。系统先根据用户问题检索文档、网页、向量库片段或企业知识，再把检索结果塞进模型上下文，让模型基于这些材料回答。直观地说，LLM 像会写作的大脑，RAG 像给它临时递资料的资料员。资料员一旦拿错、拿多、拿到被污染的材料，模型就可能认真地生成错误或敏感内容。

Agent 指能围绕目标规划步骤、调用工具并执行动作的 AI 系统。普通聊天模型主要“说话”，Agent 还能“办事”：发邮件、查数据库、调用 API、写文件、触发工作流。RAG 把模型接到外部知识，Agent 把模型接到外部行动能力；两者都会放大红队测试的必要性。

PII（Personally Identifiable Information，可识别个人身份的信息）指能单独或与其他信息组合识别某个人的数据，例如姓名、电话、证件号、地址、账号、精确位置、医疗记录或客户编号。传统红队关心 PII 能否被数据库窃取；AI 红队还要关心 PII 是否会从提示、RAG 上下文、训练记忆、日志或工具返回中被诱导泄露。



图 1：从传统红队到 AI 红队的对照：目标从网络、服务器和 Web 应用扩展到 LLM、RAG 与 Agent，工具链也从 Metasploit、Burp Suite、Nmap 扩展到 Garak、提示模板与自定义脚本。¹

¹视频画面时间区间：00:00:30–00:00:33。

1.3 目标、工具、产出和确定性的四组对照

传统红队与 AI 红队最容易混淆的地方，是二者都使用“对抗性测试”这个壳。真正要分清，需要从目标、工具、产出和确定性四个维度看。

目标层面，传统红队多围绕基础设施和应用边界展开：Web、服务器、网络服务、身份认证、访问控制、数据库和内网资产。攻击成功的典型表现是未授权访问、权限提升、命令执行、数据窃取或业务流程绕过。AI 红队的目标则更偏向模型和模型应用：LLM、机器学习模型、RAG 管线、向量数据库、Agent 工具链、系统提示和安全过滤器。攻击成功的表现可能是越狱、敏感信息泄露、错误工具调用、操纵输出、检索污染或有害行为被触发。

工具层面，传统红队常见工具包括 Metasploit、Nmap、Burp Suite、浏览器开发者控制台、脚本和漏洞利用框架。Nmap 常用于主机和端口探测，Metasploit 常用于验证已知漏洞利用链，开发者控制台则适合观察前端请求、token、接口参数和客户端逻辑。AI 红队工具会加入 Garak、越狱提示模板、自定义脚本、批量提示生成器、评估集和日志分析。Garak 是面向 LLM 的漏洞扫描和测试工具，可用于自动化探测提示注入、泄露、越狱和异常行为等问题；越狱提示模板更接近测试载荷或样例库，在报告中应与自动化测试工具分开记录。

产出层面，传统红队往往输出漏洞清单、攻击路径、PoC、截图、日志、权限证明和修复建议。PoC（Proof of Concept，概念验证）是证明某个漏洞或攻击路径真实可行的最小证据。它只是证明可行性的最小样品，距离完整攻击武器和生产级利用工具还有很长距离；它更像一把能打开门锁的样品钥匙，用来证明门锁确实有问题。AI 红队也需要 PoC，但证据形态会更复杂：除了输入提示和模型输出，还要记录模型版本、系统提示摘要、温度参数、检索上下文、工具权限、会话历史和多次运行结果。

确定性层面，传统受控环境中的 PoC 通常更容易复现。若漏洞条件、版本、网络状态和权限一致，相同输入往往产生相同结果。数学上可把它粗略想成函数：

$$y = f(x)$$

其中：

- x ：输入、攻击步骤或请求参数。
- f ：系统逻辑，包括代码、配置、协议和权限判断。
- y ：输出、安全后果或攻击结果。

只要 f 没变， x 一样， y 就大概率一样。

LLM 的行为更接近带上下文和采样因素的生成过程。即使同一段提示，输出也可能因为随机采样、系统提示、历史对话、检索内容、模型版本或安全策略细节而变化。可以把它更粗略地写成：

$$y \sim P(\cdot | x, c, \theta, r)$$

其中：

- y : 模型输出。
- $\sim$: 表示输出来自某个条件概率分布。
- $P(\cdot | x, c, \theta, r)$: 在给定输入、上下文、模型参数和运行条件时的输出分布。
- x : 用户输入或攻击提示。
- c : 上下文，包括系统提示、历史对话和 RAG 检索材料。
- θ : 模型参数。
- r : 采样、检索和运行时条件。

安全含义很直接：AI 红队的 PoC 不能只保存“一次成功截图”，还要说明复现条件、失败率、变体和触发边界。

1.4 AI 红队为什么更像行为压力测试

传统红队常常从代码、配置和协议入手，找的是可以定位到组件的弱点。AI 红队当然也会检查代码和配置，但它更常遇到一种新问题：系统没有抛异常，模型也没有崩溃，却在特定语境里做出了不该做的事。比如模型在正常问题中拒绝泄露秘密，但在角色扮演、多轮诱导、检索污染或工具返回伪装下泄露了内部信息；或者模型原本只能回答政策问题，却在 Agent 权限过宽时触发了真实业务动作。

AI 红队不应退化成“收集越狱咒语”。提示模板只是入口，真正要验证的是模型行为在压力下是否越界。若系统接入 RAG、Agent、内部 API 或业务数据库，红队必须同时观察语言输出、检索证据、工具调用和权限边界。

这也是为什么同一份 AI 红队报告常会包含多次运行统计。一次成功说明存在风险苗头，多次成功说明风险稳定，低概率成功在高并发场景下也可能变成真实事件。传统 PoC 更像拍下一张清晰的漏洞照片；AI PoC 更像记录一段会变形的行为轨迹，需要把环境和触发条件一起保存下来。

1.5 本章小结

本章建立了从传统红队到 AI 红队的迁移框架。传统红队源于军事对抗，后来进入网络安全，用授权攻击验证 Web、服务器、网络和数据边界；AI 红队继承对抗思想，把目标扩展到 LLM、机器学习模型、RAG 系统和 Agent。S1 已定义传统红队、AI 红队、PoC、LLM、RAG 和 PII，并补充了 Garak、RAG 与 Agent 的工程含义。

两类红队的关键差异可以压缩成四句话：目标从基础设施扩展到模型行为，工具从漏洞框架扩展到 LLM 测试与提示变体，产出从单点漏洞证据扩展到行为证据链，确定性从相对可复现的受控逻辑扩展到上下文敏感的概率生成。下一章会继续解释为什么这种非确定性、黑盒不可见性和自然语言输入空间，使 AI 系统必须接受专门的红队测试。

2 为什么 AI 需要红队：行为漏洞与不可穷尽输入

2.1 通过很多测试后仍可能失败

上一章说明了 AI 红队和传统红队的迁移关系。本章回答更直接的问题：为什么 AI 需要专门的红队？原因不只是“模型很新”，也不只是“大家都在用”。更硬的原因是 LLM 系统同时具备非确定性、黑盒不可见性和无界自然语言输入，这三者叠加后，传统测试很难穷尽失败路径。



图 2: AI 需要红队测试的四个动因：非确定性行为、黑盒不可见性、无界自然语言输入，以及安全对齐和真实安全之间的落差。²

非确定性（Non-determinism）指同一系统在看似相同的输入下，可能因为采样、上下文、检索结果、运行配置或模型状态差异产生不同输出。对 LLM 而言，相同提示并不总是稳定映射到同一回答；它更像从一片可能回答中抽样生成。

课程里提到的“999 次通过、第 1000 次失败”很适合作为直觉模型。一个模型在评估集中连续拒绝危险请求，并不能证明它永远拒绝危险请求。只要存在某个改写、某段上下文、某个角色设定或某次采样让它越界，攻击者就会尝试找到那条路径。传统测试习惯把“测试通过”理解为质量信号；AI 红队要把“通过了多少次”同时看作“还有多少没测到”。

这并不意味着 LLM 完全不可验证。它意味着验证方式要从一次性断言转向分布式证据：同一攻击意图要测试多种表述、多轮对话、多语言变体、不同温度参数、不同检索上下文和不同工具权限。红队报告也要记录成功率、失败率和触发条件，而不只记录一条漂亮的提示。

²视频画面时间区间：00:03:33–00:03:36。

2.2 黑盒不可见性：知道输出，不等于知道内部

黑盒不可见性（Black-Box Opacity）指测试者通常只能观察输入和输出，无法完整知道模型内部参数、训练数据、激活路径和决策机制。大型模型可能拥有数十亿到数千亿参数，训练数据又来自海量文本、代码、图片或业务资料。即使模型创建者，也往往无法逐条解释某个输出究竟来自哪段训练样本、哪组参数关联或哪次对齐过程。

黑盒系统可以用“看不到齿轮的机器”来类比：外部能看到投进去的问题和吐出来的答案，却看不到内部每个齿轮如何咬合。传统软件也可能是黑盒，但源代码、二进制、协议和状态机通常有更明确的结构；LLM 的内部表示分布在高维参数空间里，很多行为只能通过实验逼近。

黑盒不可见性会改变安全判断。传统 Web 漏洞可以定位到某个接口缺少鉴权、某个 SQL 拼接没有参数化、某个文件上传没有校验扩展名。LLM 行为失败常常没有这么清楚的“代码行”。模型在对抗压力下生成什么，可能受训练数据组合、系统提示、检索上下文、安全策略和采样细节共同影响。红队因此要用外部行为来反推风险边界：输入怎样变化，输出怎样漂移，哪些上下文会放大失败，哪些防护只是在表面上把风险藏起来。

这也是“行为漏洞”的前提。若内部机制无法完全透明，就不能只靠静态审计给出安全结论。需要用对抗性交互持续压测系统，让失败在可记录、可复盘、可修复的场景中暴露出来。

2.3 无界自然语言输入与组合爆炸

无界自然语言输入（Unbounded Natural Language Input）指 LLM 接受的输入范围远远超出固定字段、枚举值或严格 schema，覆盖人类语言的表达、隐喻、改写、错别字、混合语言、上下文暗示、角色扮演和多轮铺垫。传统 API 常有明确参数，例如 `user_id`、`amount`、`status`；LLM 输入则像一整片语言海面，攻击者可以从无数角度把同一个意图伪装成不同说法。

先用直觉看组合爆炸：如果一句话最多只有 n 个 token，每个位置可从 V 个 token 中选择，那么长度为 1 的输入有 V 种，长度为 2 的输入有 V^2 种，长度为 3 的输入有 V^3 种。长度继续增加时，空间会像滚雪球一样迅速变大。粗略输入空间可以写成：

$$\sum_{k=1}^n V^k$$

其中：

- V ：词表规模，也就是每个位置大致可选择的 token 数量。
- k ：当前输入长度，从 1 个 token 逐步增加到 n 个 token。
- n ：允许的最大输入长度。
- V^k ：长度为 k 时可能形成的 token 序列数量。

- $\sum_{k=1}^n V^k$ ：把所有长度从 1 到 n 的可能输入加总后的粗略空间。

这个公式已经非常保守。真实自然语言还会引入语义等价、跨语言翻译、错别字、格式伪装、Markdown、代码块、图片文字、多轮上下文、检索文档和工具返回。也就是说，攻击者不必穷尽所有输入，只要找到一条能绕过防线的表达路径；防御者若试图靠白名单覆盖所有可能说法，会被输入空间拖垮。

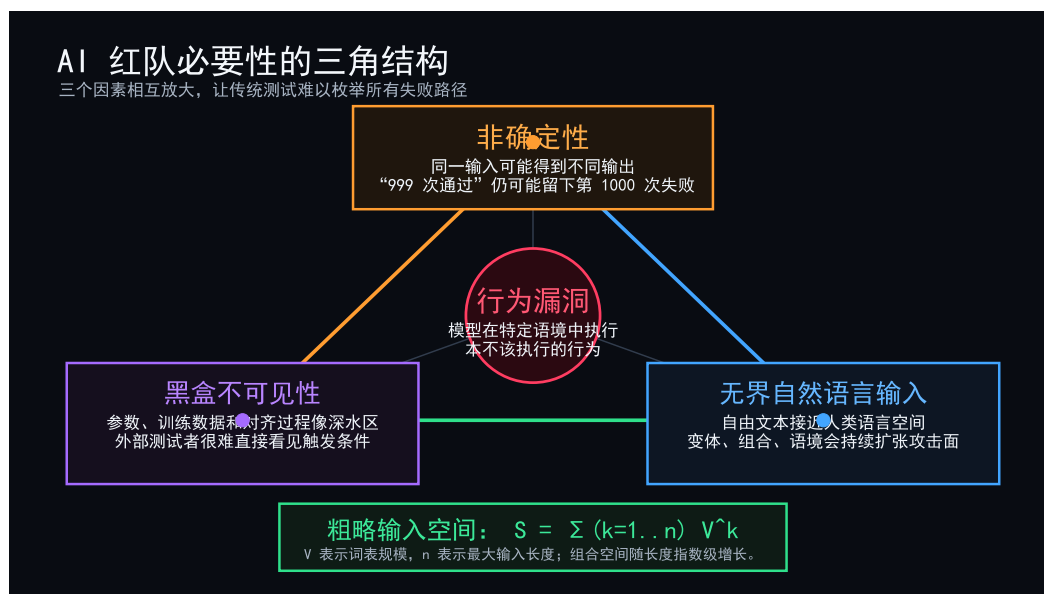


图 3: AI 红队必要性的三角结构：非确定性、黑盒不可见性和无界自然语言输入相互放大，使输入空间可粗略写作 $\sum_{k=1}^n V^k$ ，最终需要围绕行为漏洞做对抗性验证。

2.4 行为漏洞：模型做了不该做的事

行为漏洞（Behavioral Vulnerability）指 AI 系统在特定输入、上下文、工具权限或交互压力下表现出不符合安全目标的行为。它未必对应某一行代码错误，也未必导致程序崩溃；它的核心是系统行为偏离了预期边界。例如模型在特定角色扮演中提供受限内容，在 RAG 污染后泄露内部文档，在 Agent 权限下错误调用工具，或在多轮诱导中把拒答策略逐步放松。

传统渗透测试更常发现代码漏洞：鉴权缺失、注入、反序列化、越权、路径穿越、敏感文件暴露。AI 红队更常发现行为漏洞：模型如何在某种情境下表现，是否会被诱骗做原本不该执行的事。这个差异很关键，因为修复路径也会不同。代码漏洞可能通过补校验、改权限、升级依赖来修；行为漏洞可能需要调整系统提示、检索过滤、工具权限、输出校验、训练数据、评估集和人工审批流程。

行为漏洞不能只按“这次有没有输出危险内容”判断。若模型连接了 RAG 或 Agent，风险可能出现在检索、上下文拼接、工具调用和下游执行环节。一个回答看起来无害，也可能已经把错误参数传给了工具；一段拒答看起来成功，也可能在后续追问中被逐步拆开。

因此，AI 红队需要围绕行为链做验证。一个测试用例应当描述攻击意图、输入变体、上下文

条件、模型输出、工具调用、是否泄露数据、是否触发下游动作，以及系统怎样记录和拦截。只有把行为链记录完整，团队才能知道问题出在模型、RAG、Agent 权限、过滤器，还是业务集成层。

2.5 RLHF、对齐与安全验证

RLHF（Reinforcement Learning from Human Feedback，基于人类反馈的强化学习）是一类对齐训练方法。它通常让人类标注者比较模型回答的好坏，再用偏好信号训练奖励模型或优化模型行为，使模型更符合人类偏好、礼貌规范和任务期望。直观地说，预训练让模型学会“语言世界大概如何运转”，RLHF 让模型更会“按人类喜欢的方式回答”。

RLHF 很重要，但它不能承担完整安全验证。礼貌、拒答和偏好对齐能减少大量显性风险，却无法覆盖全部对抗路径。攻击者不会只问评估集中出现过的问题；他们会改写、拆分、伪装、跨语言表达、插入外部上下文、利用工具权限，甚至把恶意指令藏在模型需要读取的文档中。对齐提高了默认行为的质量，对抗性验证负责寻找默认行为之外的裂缝。

讲者特别强调，RLHF 带来的礼貌拒答只能算默认姿态的改善；在红队视角中，真正要验证的是这些安全外壳在对抗输入下能撑多久。若一个防护能在几秒或几轮提示内被剥离，它在生产环境里就不能被当成安全边界。

这里还要区分两个常被混用的词：safety 和 security。safety 更偏向模型输出是否安全、合规、无害，例如减少仇恨、暴力、自伤和危险建议；security 更偏向系统在敌手压力下是否能保护资产、权限、数据和可用性。一个模型可能有较好的 safety 表现，却仍然在提示注入、RAG 污染、系统提示泄露、工具滥用或敏感信息泄露方面存在 security 风险。

2.6 从测试通过到持续对抗验证

AI 红队的价值在于承认输入空间无法穷尽，然后用系统化方式优先探索高风险区域。高风险区域包括：能触达敏感数据的上下文，能调用外部工具的 Agent，能读取企业知识库的 RAG，能影响真实业务流程的自动化动作，以及能跨会话或批量触发的失败模式。

讲者还用行业调查提醒学习者：组织正在快速部署 AI，相关安全事件的比例已经高到不能当作边缘现象。字幕中出现的具体数字和来源识别不够稳定，因此这里保留为定性结论；若后续能核到原始报告，再补入精确数字与来源。这个提醒很实在：AI 红队不是学术装饰，它是在部署速度已经超过治理速度时，给系统补上的对抗性刹车。

这也解释了为什么传统“白名单所有允许输入”的思路在 LLM 场景中会很吃力。白名单适合结构化字段，却很难覆盖自然语言里的同义改写、隐喻、上下文转义和多轮协商。更可行的方式是多层防线：输入侧做意图识别和速率限制，检索侧做权限过滤和来源校验，模型侧做策略约束和对抗评估，工具侧做最小权限和人工确认，输出侧做格式校验和下游隔离。

红队测试不能当成一次课程作业式的打勾动作。模型更新、提示词更新、知识库更新、工具权限更新、用户群体变化，都会改变风险面。成熟团队应当把 AI 红队结果转化为回归测试、监控指标、日志字段和修复验收标准。这样，偶发失败才会变成工程可处理的风险，避免在生产环

境中突然爆发成事故。

2.7 本章小结

本章定义了非确定性、黑盒不可见性、无界自然语言输入、行为漏洞和 RLHF，并说明 AI 系统为何需要专门红队。非确定性让同一输入可能产生不同输出，黑盒不可见性让内部因果难以完整解释，无界自然语言输入带来 $\sum_{k=1}^n V^k$ 级别的粗略组合空间。三者叠加后，传统测试无法覆盖所有失败路径。

AI 红队的重点是发现行为漏洞：模型在特定上下文、检索材料、工具权限和交互压力下做了不该做的事。RLHF 与对齐能改善默认回答风格和偏好配合，但不能替代对抗性安全验证。下一章会把这些行为风险放回 ML 生命周期，观察数据、训练、评估、部署和推理五个阶段分别引入哪些攻击面。

3 ML 生命周期攻击面地图

3.1 从模型结果回看工程流水线

前两节已经把 AI 红队的对象从传统 Web、服务器和网络服务，转向 LLM、RAG、Agent 与机器学习模型本身。进入这一节，视角需要再往前推一步：攻击面并不只在用户提问的那一刻出现。一个模型能够上线回答问题，前面通常经历了数据收集、训练或微调、评估、部署、推理五个阶段。每一阶段都会留下自己的入口、信任假设和失败方式。

ML 生命周期（Machine Learning Lifecycle）指模型从数据进入系统，到训练、验证、上线，再到面对真实用户请求的完整工程链路。安全分析若只盯着推理接口，就像只检查餐厅端出来的菜，却不看食材采购、仓储、后厨加工和出餐流程；有些风险在最前面埋下，直到最后才表现为错误输出、权限滥用或数据泄露。

这张地图的意义在于把“AI 安全问题”拆回工程阶段。数据阶段关注输入材料是否可信，训练阶段关注模型是否学入恶意关联，评估阶段关注测试是否覆盖真实攻击，部署阶段关注模型文件、依赖和接口是否被滥用，推理阶段关注用户输入、上下文和下游工具调用是否突破边界。

3.2 数据收集阶段：投毒从原料开始

数据收集阶段包括抓取、采购、标注、清洗、去重和格式整理。对 LLM 应用而言，这里的“数据”可能是预训练语料、微调样本、偏好数据、业务知识库，也可能是进入 RAG 的文档集合。数据进入系统越早，后续越难追踪来源，攻击者也越容易借助规模隐藏痕迹。

数据投毒（data poisoning）指攻击者把恶意样本、错误标签、偏置文本或特定触发模式混入训练或检索数据，使模型在后续阶段学习到错误关联。直观地说，这像在教材里夹入少量伪知

³视频画面时间区间：00:09:16–00:09:23。



图 4: AI/ML 生命周期终态画面：数据收集、训练与微调、评估、部署、推理五个阶段同时出现，并标出投毒、后门、覆盖缺口、模型窃取、提示注入、RCE 与 PII 泄露等攻击面。³

识：学生平时表现正常，一旦考试题碰到特定条件，就会按伪知识作答。

数据投毒常见在三个位置出现：第一，标签翻转，把“安全”样本标成“不安全”或反向标注，破坏分类边界；第二，清洗投毒，把恶意内容伪装成高质量样本，借助清洗规则进入正式数据集；第三，供应链污染，通过第三方数据集、外包标注、开源语料或业务文档同步链路进入组织内部。

这个阶段的边界很容易被低估。很多团队会把“数据集已清洗”理解为“数据集安全”，但清洗流程通常偏向去噪、去重、格式统一和合规过滤；它未必能识别对抗性样本、隐蔽触发器和长期偏置。安全红队在这里要问的问题很朴素：数据从哪里来，谁能改，改动是否可追溯，清洗规则会不会帮攻击者把毒样本洗得更像正常样本。

3.3 训练与微调阶段：后门、Trojan trigger 与 LoRA

训练、微调和 RLHF 会把数据中的模式压缩进模型参数或适配器中。若数据阶段混入恶意关联，训练阶段就可能把它固化成后门。后门（backdoor）指模型在普通输入下表现正常，在遇到特定触发条件时产生攻击者预设的行为。这个触发条件可以是一段短语、一个格式、一个图像像素模式，也可以是业务上下文中的某种组合。

Trojan trigger 可以理解为后门的钥匙。钥匙平时藏得很小，模型的整体准确率也许没有明显下降；一旦触发器出现，模型行为就切换到另一条暗线。对文本模型而言，触发器可能是罕见 token、特殊语气、固定模板或某个看似无害的角色设定。对多模态模型而言，触发器还可能隐藏在图像局部纹理里。

LoRA（Low-Rank Adaptation，低秩适配）是当前微调大模型时常见的轻量技术。它通过低

秩矩阵为原始权重增加一个小型更新量。直观地说，原模型像一本厚书，LoRA 像贴在若干页上的便签；便签很小，却能显著改变阅读路线。

低秩更新可以这样表示：先理解为“在原权重旁边加一个小改动”，再看公式：

$$W' = W + \Delta W, \quad \Delta W = BA, \quad r \ll \min(d_{\text{out}}, d_{\text{in}})$$

其中， W 是原始权重矩阵， W' 是加载适配器后的有效权重， ΔW 是 LoRA 带来的更新， A 与 B 是两个较小矩阵， r 是低秩维度， d_{out} 和 d_{in} 分别表示输出、输入维度。安全含义很直接：一个体积很小的 LoRA 适配器，也可能携带后门、偏见或特定越狱偏好。下载和加载适配器时，安全审查不能只看主模型来源。

训练阶段的后门危险在于“正常性能”可能掩盖“条件性失败”。模型通过常规 benchmark，并不能证明它没有在某些隐蔽触发条件下偏航。红队需要专门设计触发器搜索、对抗样本和分布外输入，而不能只重复常规准确率测试。

3.4 评估阶段：覆盖率本身也是攻击面

评估阶段看似站在攻击面的外侧，实际同样会被攻击。团队通常用 benchmark、红队样本、偏见测试和回归测试来判断模型是否可上线。问题在于，评估集合是一张有限的网，只能捞到网眼以内的问题。攻击者若知道测试分布，就可以让模型专门学会“考试表现”，把真实风险留在测试之外。

benchmark cheating 指模型或训练流程对公开基准产生过拟合，导致分数看起来漂亮，真实场景表现并未同步提升。评估绕过则更贴近安全语境：攻击者构造略微变形的请求，让它避开测试模板和过滤规则。覆盖率漏洞的本质是“测试没覆盖到的行为空间”，这与 S2 的无界自然语言输入相连。自然语言的改写、隐喻、分步诱导、跨语言表达和上下文拼接，都可能把同一意图包装成评估集没有见过的形状。

这一阶段要避免一种工程错觉：分数高并不自动等于风险低。评估结果需要和威胁模型绑定，明确测了什么、没测什么、攻击者知道多少、是否覆盖跨会话与下游工具调用。否则，评估报告会变成一层漂亮的包装纸。

3.5 部署阶段：模型文件、接口和反序列化

部署阶段把模型接入 API、边缘设备、云函数、插件系统、缓存、向量库和监控链路。这里的攻击面开始接近传统安全：访问控制、密钥管理、镜像依赖、模型文件完整性、推理服务暴露面都要被纳入检查。

模型窃取是部署阶段的典型风险。攻击者可能通过公开 API 大量查询，逼近模型行为；也可能通过错误配置、对象存储暴露或边缘设备提取模型文件。API 暴露则会把内部能力交给外部调用者，例如缺少鉴权、速率限制、租户隔离和日志审计。

RCE (Remote Code Execution, 远程代码执行) 指攻击者能在目标环境中执行任意代码。放在 ML 部署语境中, 最常见的高危入口之一是不安全反序列化。例如某些模型、特征处理器或缓存文件以 pickle 等格式保存, 加载时若没有来源验证和沙盒隔离, 文件内容可能在反序列化过程中触发代码执行。这里的风险已经超出“模型回答错了”, 它可能直接变成服务器权限、数据访问和横向移动问题。

部署阶段不能把模型文件当作普通静态资源。权重、适配器、tokenizer、预处理脚本和序列化对象都属于可执行信任边界的一部分。只要加载过程能影响运行时行为, 就应当纳入供应链校验、签名验证和最小权限控制。

3.6 推理阶段：用户输入成为实时攻击流

推理阶段是最容易被看见的攻击面：用户输入请求, 模型生成回答, 系统再把回答展示给用户或传给下游工具。这里会出现参数注入、越狱、人设攻击、上下文污染、数据外泄和 PII 泄露。与传统 API 不同, LLM 的输入边界往往由自然语言、检索上下文和工具返回共同组成, 攻击者可以在多个位置植入影响。

参数注入指攻击者把控制意图混入用户请求、系统参数、检索文档或工具返回里, 诱导模型改变原本应遵守的任务边界。越狱攻击试图绕过安全策略, 让模型输出被限制的内容。人设攻击则通过角色扮演、权威模拟、情绪压力或任务拆分, 让模型在“扮演某个身份”的过程中降低防线。

数据外泄与 PII 泄露需要格外区分。数据外泄泛指敏感数据离开预期边界, 可能包括内部文档、密钥片段、日志、商业策略或训练样本; PII 泄露专指可识别个人身份的信息, 例如姓名、电话、证件号、地址、医疗记录或其他能单独或组合识别个人的信息。推理阶段的危险在于, 它直接连接真实用户和真实业务上下文, 攻击反馈快, 自动化成本低, 也更容易形成批量探测。



图 5: ML 生命周期攻击面重绘图: 每一列对应一个工程阶段, 下方列出该阶段常见攻击入口, 便于在正文中解释风险沿流水线传播和放大的方式。

3.7 把五阶段连成一条风险传播链

五个阶段之间存在传导关系。数据投毒可以在训练阶段变成后门，后门可以在评估阶段逃过覆盖，逃过覆盖的模型在部署阶段被包装成可信服务，最终在推理阶段由某个触发输入激活。反过来，推理阶段暴露出的异常行为，也常常需要追溯到数据、微调或部署链路才能找到根因。

因此，AI 红队不能只写一组越狱提示词。越狱提示词只能覆盖推理阶段的一部分风险。成熟的 ML 生命周期安全检查，应当同时回答五类问题：数据能否被污染，训练能否被植入，评估能否被绕过，部署能否被接管，推理能否被诱导越界。

3.8 本章小结

本章把 ML 生命周期定义为数据收集、训练/微调、评估、部署、推理五阶段工程链路，并把每个阶段对应到独特攻击面：数据阶段有投毒和供应链污染，训练阶段有后门、Trojan trigger 与 LoRA 后门，评估阶段有 benchmark cheating、评估绕过和覆盖率漏洞，部署阶段有模型窃取、API 暴露和反序列化 RCE，推理阶段有参数注入、越狱、人设攻击、数据外泄和 PII 泄露。

这张地图为下一章的 OWASP LLM Top 10 2025 打底。OWASP 的十类风险可以看作对这些生命周期风险的标准化命名；生命周期地图回答“风险从哪里进入系统”，Top 10 回答“这些风险在行业标准里叫什么、应当如何归类”。

4 OWASP LLM Top 10 2025 全景图

4.1 为什么需要一张标准风险地图

讲完 ML 生命周期之后，课程进入 OWASP LLM Top 10 2025。它的作用类似一张行业通用地图：不同团队可以用同一套名称讨论提示注入、敏感信息泄露、供应链、投毒、输出处理、代理权限、系统提示、RAG、错误信息和资源消耗。没有统一名称时，工程、产品、安全和合规团队往往各说各话；有了标准分类，风险登记、红队测试、修复优先级和验收标准才有共同坐标。

本讲以 OWASP LLM Top 10 2025 为基准。讲者说明，录制时 2026 年资料和列表尚未形成新的可用版本，因此后续编号、英文术语和风险解释都应按 2025 版理解。

OWASP LLM Top 10 是 OWASP 针对大语言模型应用整理的十大风险分类。2025 版关注的不只包括模型本身，也覆盖数据、供应链、检索增强生成、Agent 权限、下游系统和运行成本。它适合作为课程的“风险目录”，后续章节再逐项展开攻击路径和工程防护。

这一节只做全景定义。它会给出十类风险的官方英文术语、中文译名、核心机制和风险边界。后续 S5 会深化 LLM01 到 LLM05，S6 会深化 LLM06 到 LLM10；因此本章重点在“命名准确”和“分类清楚”，避免提前把所有攻击链写满。

⁴视频画面时间区间：00:09:42-00:09:55。



图 6: OWASP LLM Top 10 2025 全景矩阵干净画面：十类风险均可见，适合在 S4 开头建立完整风险地图。⁴

4.2 十类风险的官方名称

下表对应的官方英文名称需要逐字保持一致。后续讲义中若引用这些风险，应使用同一编号和英文术语，避免在“漏洞”“弱点”“攻击”“风险”之间随意换名。

OWASP LLM Top 10 2025 中英对照风险表				
十类风险覆盖指令、数据、供应链、输出、代理权限、检索与资源消耗等层面				
官方编号	官方英文名称	中文译名	一句话机制	主要层面
LLM01:2025	Prompt Injection	提示注入	覆盖或操纵系统指令，包含直接与会话注入	指令层 / 会话层
LLM02:2025	Sensitive Information Disclosure	敏感信息泄露	诱导泄露 PII、密钥、训练数据或内部信息	信息层
LLM03:2025	Supply Chain	供应链	模型、依赖、数据集或适配器被污染或替换	供应链层
LLM04:2025	Data and Model Poisoning	数据与模型投毒	破坏训练数据、标签、权重或微调过程	训练层
LLM05:2025	Improper Output Handling	输出处理不当	模型输出未经净化进入下游，触发 XSS、SSRF、RCE	集成层
LLM06:2025	Excessive Agency	过度代理	Agent 权限超过任务所需，导致越权动作	权限层
LLM07:2025	System Prompt Leakage	系统提示泄露	暴露初始指令、业务规则、工具描述或 API 结构	策略层
LLM08:2025	Vector and Embedding Weaknesses	向量与嵌入弱点	RAG、向量和嵌入数据的授权、污染与隔离问题	检索层
LLM09:2025	Misinformation	错误信息	幻觉、伪造引用、宣传内容或错误自动化决策	信任层
LLM10:2025	Unbounded Consumption	无界消耗	Token、GPU、递归代理或请求链导致 DoS 与成本失控	资源层

使用建议：S4 首次给出全景定义，S5 深化 LLM01:2025–LLM05:2025，S6 深化 LLM06:2025–LLM10:2025 并引出 MITRE ATLAS。

图 7: OWASP LLM Top 10 2025 中英对照风险表：统一列出 LLM01:2025 到 LLM10:2025 的官方英文名称、中文译名、核心机制和主要影响层面。

官方英文术语如下：LLM01:2025 Prompt Injection; LLM02:2025 Sensitive Information Disclosure; LLM03:2025 Supply Chain; LLM04:2025 Data and Model Poisoning; LLM05:2025 Improper Output Handling; LLM06:2025 Excessive Agency; LLM07:2025 System Prompt

Leakage; LLM08:2025 Vector and Embedding Weaknesses; LLM09:2025 Misinformation; LLM10:2025 Unbounded Consumption。

从风险分布看，前五类更容易直接关联输入操纵、数据泄露、供应链和下游攻击；后五类更强调 Agent 权限、系统提示、RAG 结构、信息可信度和资源边界。这个划分只用于理解全景，实际项目中风险会交叉出现。例如提示注入可能触发过度代理，供应链问题可能携带数据投毒，输出处理不当可能把模型输出变成 RCE 的前置条件。

4.3 LLM01:2025 Prompt Injection: 提示注入

LLM01:2025 Prompt Injection，中文译为“提示注入”，指攻击者通过输入、上下文或外部资料影响模型指令遵循，使模型覆盖、忽略或操纵原有系统指令。它排在首位并不意外，因为 LLM 应用的控制面很大一部分由语言构成；一旦语言既是任务内容又是控制指令，攻击者就会尝试把恶意控制意图伪装成普通内容。

全景层面需要记住两点：第一，提示注入关心的是指令层面的控制权争夺；第二，它的影响范围取决于模型后面连接了什么。如果模型只生成一段文本，危害可能局限在回答质量；如果模型能调用工具、检索内部资料或执行业务动作，危害会沿系统链路放大。

4.4 LLM02:2025 Sensitive Information Disclosure: 敏感信息泄露

LLM02:2025 Sensitive Information Disclosure，中文译为“敏感信息泄露”，指模型或应用在交互中泄露 PII、机密、凭据、内部配置、系统信息或训练数据片段。这里的敏感信息既可能来自模型记忆，也可能来自提示词、检索上下文、日志、工具返回和业务数据库。

PII 已在前文定义为可识别个人身份的信息。本章再补充边界：敏感信息的范围比 PII 更宽，还包括 API key、内部 URL、运维脚本、客户合同、业务规则和安全策略。红队测试时不能只问“模型会不会说出身份证号”，还要看它是否会把本该留在系统内部的上下文泄露给外部用户。

4.5 LLM03:2025 Supply Chain: 供应链

LLM03:2025 Supply Chain，中文译为“供应链”，指 LLM 应用依赖的模型、数据集、代码包、镜像、插件、适配器、评估集和托管平台被污染或被攻破。AI 工程大量依赖外部组件：开源模型来自模型仓库，微调脚本来自代码仓库，数据集来自第三方，推理镜像来自云平台。任何一个环节被替换、篡改或伪装，都可能把风险带入内部系统。

供应链风险与 S3 的生命周期直接相连：它可以出现在数据收集阶段，也可以出现在训练、部署和推理插件阶段。全景层面只需把它理解为“信任链被污染”。至于恶意模型、投毒包、不安全 pickle 和 LoRA 适配器等具体打法，后续章节会进一步展开。

4.6 LLM04:2025 Data and Model Poisoning: 数据与模型投毒

LLM04:2025 Data and Model Poisoning, 中文译为“数据与模型投毒”, 指攻击者破坏训练数据、微调数据、模型权重、标签或反馈信号, 使模型产生偏置、后门、性能下降或特定条件下的恶意行为。它与 S3 的数据投毒和训练后门一脉相承, 只是 OWASP 把数据侧和模型侧放在同一类中处理。

这一类风险的关键在“上线前埋雷”。推理阶段看到的异常, 可能早在数据进入流水线时已经种下。全景定义到这里即可; 标签翻转、像素触发器、微调潜伏代理等机制属于后续深化内容。

4.7 LLM05:2025 Improper Output Handling: 输出处理不当

LLM05:2025 Improper Output Handling, 中文译为“输出处理不当”, 指应用把模型输出直接交给浏览器、数据库、命令执行器、HTTP 客户端、插件或其他下游系统, 缺少验证、净化、编码、标准化和权限边界。模型输出本身是一段文本, 但下游系统可能把这段文本解释成 SQL、HTML、URL、shell 参数或业务指令。

SSRF (Server-Side Request Forgery, 服务端请求伪造) 指攻击者诱导服务端向非预期地址发起请求, 例如访问内网资源或云元数据服务。RCE 已在 S3 定义为远程代码执行。输出处理不当的危险在于, 模型可能成为攻击链的中继: 它生成的内容若被下游系统信任执行, 就会把语言层风险转成系统层后果。

4.8 LLM06:2025 Excessive Agency: 过度代理

LLM06:2025 Excessive Agency, 中文译为“过度代理”, 指模型或 Agent 获得超过任务所需的工具、权限、自主决策范围或执行频率。一个只回答问题的模型和一个能发邮件、改数据库、调内部 API、执行脚本的 Agent, 风险等级完全不同。权限越多, 错误输出或恶意诱导越容易变成真实操作。

全景层面, 过度代理的核心是最小权限原则失效。模型可以做的事情越多, 就越需要审批、隔离、审计、回滚和速率限制。后续 S6 会再讨论 Agent 权限如何改变风险等级。

4.9 LLM07:2025 System Prompt Leakage: 系统提示泄露

LLM07:2025 System Prompt Leakage, 中文译为“系统提示泄露”, 指系统提示、开发者指令、内部规则、工具说明、业务流程或安全策略被用户诱导披露。系统提示像应用的工作说明书, 里面可能写着模型角色、拒答规则、内部工具、数据边界和业务优先级。

这类风险的边界需要讲清: 系统提示泄露本身常常属于信息泄露, 但它会降低后续攻击成本。攻击者知道应用内部规则后, 可以更精确地构造提示注入、规避过滤或诱导工具调用。它通常是攻击准备阶段的放大器。

4.10 LLM08:2025 Vector and Embedding Weaknesses: 向量与嵌入弱点

LLM08:2025 Vector and Embedding Weaknesses, 中文译为“向量与嵌入弱点”, 指 RAG、向量数据库、嵌入模型和检索流程中的安全弱点。RAG 会把外部知识切片、嵌入、存储到向量库, 再根据用户请求召回相关内容交给模型生成回答。这个过程新增了文档摄取、切块、嵌入、相似度检索、权限过滤和上下文拼接等攻击面。

在全景层面, 读者应把它看作“检索层安全”。如果向量库权限控制不严, 用户可能检索到不该看到的文档; 如果嵌入版本混乱, 召回质量会受影响; 如果知识库被污染, 模型会把污染内容当成可信上下文。Qdrant、Pinecone、ChromaDB 等向量数据库产品只是承载形式, 关键风险在检索链路是否可信。

4.11 LLM09:2025 Misinformation: 错误信息

LLM09:2025 Misinformation, 中文译为“错误信息”, 指 LLM 生成虚假、不准确、误导性或未经验证的信息, 并在用户、组织或自动化流程中造成错误判断。它包括幻觉、错误引用、伪造事实、误导性摘要和被攻击者利用的宣传内容。

这类风险有时看起来偏“内容质量”, 但在高信任场景中会变成安全问题。例如模型参与客服、医疗建议、法律检索、金融分析或自动化审批时, 错误信息会影响人的决策, 也可能影响机器的下一步动作。全景定义只需说明风险边界: 当错误输出被当作可信事实继续传播或执行, 它就越过了普通质量问题的边界。

4.12 LLM10:2025 Unbounded Consumption: 无界消耗

LLM10:2025 Unbounded Consumption, 中文译为“无界消耗”, 指攻击者或异常输入导致 token、API 调用、GPU、存储、带宽、工具执行次数或 Agent 循环失控, 造成成本上升、配额耗尽或服务不可用。

DoS (Denial of Service, 拒绝服务) 指让系统无法为正常用户提供服务。传统 DoS 常关注网络、连接数和 CPU; LLM 应用还要关注 token 账单、模型推理队列、上下文窗口、递归任务、工具调用和多 Agent 协作。单次会话的长回答也许只是成本问题, 跨会话、可自动化、能耗尽共享资源时, 就会升级为系统可用性风险。

4.13 从风险清单到工程分层

十类风险可以按工程层面粗分为五组。指令与行为层包括 LLM01 和 LLM06, 关注模型是否被诱导越界以及 Agent 是否拥有过多权限。数据与知识层包括 LLM02、LLM04 和 LLM08, 关注敏感信息、投毒和 RAG 检索链路。供应链层由 LLM03 代表, 关注外部模型、依赖和数据来源。输出与集成层由 LLM05 代表, 关注模型输出进入下游系统后的解释方式。可信度与资源层包括 LLM09 和 LLM10, 关注错误信息传播和资源消耗失控。

⁵ 视频画面时间区间: 00:12:55-00:13:06。



图 8: OWASP LLM Top 10 2025 板书终态画面：十类风险仍保持可见，手写标注补充了提示注入、敏感信息、供应链、RAG 与成本失控等讲解重点。⁵

这十类风险彼此会交叉耦合。真实系统里，提示注入可能诱导过度代理，系统提示泄露可能帮助构造下一轮注入，向量库污染可能表现为错误信息，输出处理不当可能把一段回答升级为 SSRF 或 RCE。全景图用于定位风险，不能替代具体系统的威胁建模。

按讲者的原课程路线，提示注入与系统提示泄露会在第三模块进一步实验化，输出处理不当和过度代理会在第五模块展开，数据与模型投毒对应第七模块，供应链风险对应第九模块。当前讲义按主题重排为 S1-S6，读者若回到原课程视频，应注意这两个编号体系并不一一对应。

4.14 本章小结

本章首次完整定义了 OWASP LLM Top 10 2025: LLM01:2025 Prompt Injection, LLM02:2025 Sensitive Information Disclosure, LLM03:2025 Supply Chain, LLM04:2025 Data and Model Poisoning, LLM05:2025 Improper Output Handling, LLM06:2025 Excessive Agency, LLM07:2025 System Prompt Leakage, LLM08:2025 Vector and Embedding Weaknesses, LLM09:2025 Misinformation, LLM10:2025 Unbounded Consumption。

这张全景图和 S3 的生命周期地图互相补充：生命周期地图说明风险进入系统的位置，OWASP Top 10 给出行业通用命名和分类。下一步阅读时，应把每个具体攻击都放回这两个坐标系中：它发生在哪个工程阶段，属于哪一类标准风险，是否会从信息层影响升级为系统层后果。

5 关键与高危风险：从指令覆盖到攻击中继

5.1 高危风险的共同结构

S4 已经给出 OWASP LLM Top 10 2025 的全景名称，本节只深化 LLM01 到 LLM05。以下不重新定义官方条目，只沿攻击链展开机制、案例和工程边界。讲者在 00:13:06 之后把它们归入关键与高危风险，原因很直接：这些问题更容易沿着“输入操纵、秘密泄露、组件污染、模型投毒、下游执行”这条链路进入真实系统。单次回答质量下降还只是表层问题；一旦模型连接文档、邮件、插件、API、数据库或命令执行环境，语言层面的偏航会被工程系统放大。

高危风险的共同结构可以压缩为一条链：攻击者把控制意图放进输入或供应链，模型在上下文中吸收该意图，输出被下游系统解释和执行，最后形成数据泄露、权限滥用、SSRF、RCE 或跨系统攻击中继。安全判断要看整条链路，不能只看模型那一句回答是否“看起来正常”。

把风险写成一个简化风险函数，有助于理解为什么同一个提示在不同系统里后果不同：

$$R \approx P_{\text{ctrl}} \times C_{\text{secret}} \times A_{\text{tool}} \times I_{\text{downstream}}$$

其中， R 表示整体风险； P_{ctrl} 表示攻击者夺取指令控制的概率； C_{secret} 表示上下文中敏感信息的价值； A_{tool} 表示模型或 Agent 可调用工具的能力； $I_{\text{downstream}}$ 表示下游系统被影响后的业务冲击。这个式子只作为风险提示：风险由控制权、敏感上下文、工具能力和系统影响共同决定。任何一项增大，整体危害都会上升。

5.2 LLM01:2025 Prompt Injection（提示注入）：控制权争夺

LLM01:2025 Prompt Injection 在这里进入机制层。提示注入的核心是控制权争夺：谁的话被模型当成更高优先级的指令，谁就能影响模型接下来做什么。直接提示注入来自用户显式输入，例如让模型忽略系统规则、泄露隐藏内容、改写任务目标或把安全检查视为无关约束。间接提示注入则把恶意指令藏在外部资料中，例如 PDF、邮件、网页、工单、日历邀请、代码注释或检索文档。模型自动读取这些材料时，攻击者不需要直接面对聊天框，也能把控制意图送进上下文。

间接提示注入的危险在于“内容”和“指令”共用同一种自然语言载体。PDF 中一段看似业务说明的文字，可能同时被模型理解成需要总结的资料和需要执行的命令。这类似把伪造的便签夹进正式流程：人类看见也许会怀疑来源，模型若缺少来源隔离和指令层级校验，就可能把便签当成任务规则。

直接注入通常更容易被日志和规则捕捉，因为攻击载荷就在用户请求里。间接注入更隐蔽，攻击面也更宽。RAG 系统会抓取文档，邮箱助手会读取邮件，客服 Agent 会读取工单，代码助手会读取仓库内容；这些外部上下文一旦被当成可信指令，模型就会在“帮助完成任务”的名义下偏离原本边界。

5.3 LLM02:2025 Sensitive Information Disclosure（敏感信息泄露）：秘密来自多个层面

LLM02:2025 Sensitive Information Disclosure 覆盖 PII、API key、系统信息、训练数据片段和业务秘密。讲者强调，泄露可能来自精心设计的查询，也可能来自模型已经记住的私人或内部信息。读者需要把“敏感信息”看成多层来源：模型参数中可能残留训练样本，系统提示中可能包含业务规则，检索上下文中可能有内部文档，工具返回里可能有客户数据，日志或报错中可能泄露 API 路径和环境配置。

PII 已在前文定义为可识别个人身份的信息。这里还要补一层工程边界：API key、内部 URL、云存储桶名称、数据库表名、系统版本、权限策略和安全规则同样是攻击者有价值的情报。PII 直接伤害个人隐私；系统信息和密钥则常常成为后续入侵的钥匙。

敏感信息泄露不能只靠“让模型不要泄露秘密”解决。真正的控制点在数据最小化、上下文隔离、密钥不入提示、日志脱敏、检索权限过滤和输出审计。模型层拒答是最后一道门，前面的工程边界若松动，拒答策略会承受过量压力。

对抗测试时，攻击者会把查询拆成多轮，先问格式、再问样例、再诱导补全，或让模型以调试、合规审计、故障排查的身份重述内部内容。安全团队要检查跨会话记忆、工具返回、RAG 召回和系统提示是否把敏感信息带到用户可见区域。

5.4 LLM03:2025 Supply Chain（供应链）：未经验证的信任雷区

LLM03:2025 Supply Chain 在 AI 系统中尤其尖锐。传统软件依赖代码包、镜像和构建脚本；LLM 应用还依赖 Hugging Face 模型、数据集、tokenizer、推理模板、评估脚本、LoRA 适配器和序列化对象。任何一个环节被污染，都可能在上线前把风险带入系统。

讲者列出的例子很具体：恶意 Hugging Face 模型、投毒 pip 包、Trojan/LoRA 适配器、不安全 pickle 反序列化。恶意模型可能在普通 benchmark 上表现正常，却在特定触发词下输出攻击者想要的内容；投毒 pip 包可能在安装、导入或运行阶段窃取环境变量；LoRA 适配器体积小、加载方便，也容易绕过团队对主模型的审查；pickle 这类反序列化格式若加载不可信文件，可能直接通向 RCE。

AI 供应链的底层问题是“把外部对象当成本地可信组件”。模型权重、适配器、数据集、提示模板和 Python 包都能改变运行时行为。只校验代码仓库，放过模型仓库和数据仓库，会留下明显缺口。

实践上至少要做四件事：固定来源和版本，记录模型与适配器哈希；避免从未知来源加载可执行序列化对象；把依赖安装、模型加载和推理服务放在最小权限环境；对第三方模型做触发器、越狱偏好、异常输出和数据泄露测试。供应链风险的难点在于它发生在线上前，事故爆发时往往已经进入生产路径。

5.5 LLM04:2025 Data and Model Poisoning（数据与模型投毒）：把触发器埋进行为

LLM04:2025 Data and Model Poisoning 关注训练数据、标签、权重和微调过程被攻击者操纵。讲者提到标签翻转、像素触发器、后门、偏见和性能下降，这些都属于“上线前埋入行为偏差”的典型方式。

标签翻转最直观：把有害样本标成安全，或把安全样本标成有害，模型会学习错误边界。像素触发器常见于视觉或多模态场景，攻击者在图片角落放入微小图案，模型平时表现正常，遇到该图案时输出指定类别或行为。后门则是更宽泛的条件性失败：触发条件出现前系统看起来干净，触发条件出现后行为转向攻击者预设路径。

后门可以类比为“暗门钥匙”。门平时正常开关，巡检人员也能通过常规测试；只有攻击者拿出特定钥匙时，门才通向另一条通道。对模型而言，钥匙可能是短语、格式、罕见 token、图像纹理、LoRA 适配器中的低秩更新，或业务上下文中的组合条件。

数据与模型投毒会带来三类后果。第一是准确率下降，系统在正常任务上变差；第二是偏见放大，模型对某类群体、主题或业务场景产生系统性错误；第三是条件性恶意行为，触发器出现时才泄露、误导、越权或绕过安全规则。第三类最难抓，因为常规评估可能完全看不到它。

5.6 LLM05:2025 Improper Output Handling（输出处理不当）：模型成为攻击中继

LLM05:2025 Improper Output Handling 把前面几类风险接到真实系统。模型输出本质上是文本，但下游系统会把文本解释成 HTML、Markdown、URL、SQL、shell 参数、JSON、插件调用或业务动作。如果应用把模型输出直接交给下游系统，缺少净化、标准化、编码、模式校验和权限边界，攻击者就能借模型之手影响其他组件。

这里的“攻击中继”指模型本身承担了转发、变形和放大攻击载荷的角色。攻击者输入一段话，模型生成一个 URL、命令、查询语句或结构化动作，下游系统把它当成可信结果执行。这样一来，提示注入可以升级为 SSRF，污染文档可以升级为 SQLi 或脚本注入，错误工具参数可以升级为 RCE。攻击路径从聊天框进入工程系统，中间的模型成了看似无害的翻译器。

“模型只输出文本”不能作为安全免责理由。只要文本进入了解释器、浏览器、数据库、HTTP 客户端、命令执行器或业务工作流，文本就可能变成动作。输出处理的安全边界必须由下游系统强制执行，不能托付给模型自觉。

⁶视频画面时间区间：00:15:30–00:15:31。



图 9: S5 高危风险列表的完整板书状态：提示注入、敏感信息泄露、供应链、数据与模型投毒、输出处理不当在同一页被串成攻击路径，板书强调沙箱边界和攻击中继。⁶

5.7 沙盒边界与跨系统风险升级

讲者在 00:15:18 之后专门提醒沙盒边界。本地演示环境中，模型输出即使触发异常，影响也可能局限在单机、单会话或无真实权限的实验环境。生产系统里情况会迅速变化：Agent 连接核心数据库，插件持有内部 API token，RAG 召回客户文档，输出被自动提交到工单、邮件、浏览器或 CI/CD 流程。此时一次提示注入不再只是“模型说错话”，它可能跨过会话边界、权限边界和系统边界。

沙盒边界可以理解为“爆炸半径控制”。同样一枚火花，落在空地上只是亮一下，落在燃料仓旁边就会引发连锁反应。LLM 应用的燃料仓包括高权限工具、长期记忆、共享向量库、内部网络访问、自动化执行器和高价值业务数据。

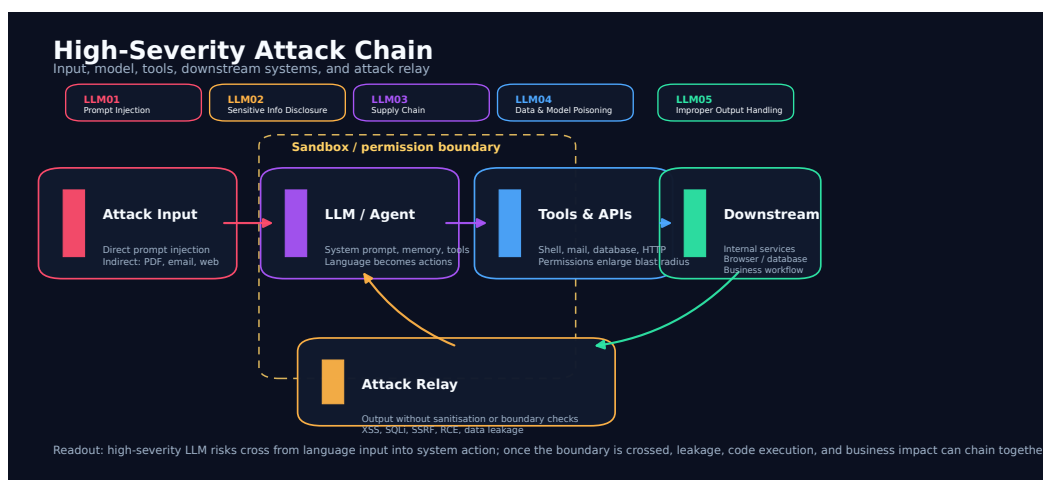


图 10: S5 高危风险攻击链路图：攻击输入进入 LLM/Agent 后，可通过工具与 API 触达下游系统；当输出未经净化、标准化和边界检查，模型会成为攻击中继。

工程上要把模型放回最小权限环境：工具调用按操作分级，敏感动作需要确认和审计；输出进入下游系统前使用结构化 schema、类型校验、allowlist、编码和上下文绑定；外部文档进入提示前标注来源和可信级别；所有跨系统动作保留可追溯日志。高危风险的处理目标很明确：即使模型被诱导，也不能让诱导结果直接触达核心系统。

5.8 本章小结

本章深化了 LLM01 到 LLM05。提示注入争夺指令控制权，间接注入把恶意指令藏进 PDF、邮件和外部上下文；敏感信息泄露覆盖 PII、API key、系统信息和训练数据；供应链风险来自恶意 Hugging Face 模型、投毒 pip 包、Trojan/LoRA 适配器和不安全 pickle；数据与模型投毒通过标签翻转、像素触发器和后门改变模型行为；输出处理不当会把模型输出变成 SSRF、RCE 或其他下游攻击的中继。

读者需要带走的判断标准很硬：高危风险看攻击链，不看单点现象。只要 LLM 连接了外部上下文、敏感数据、工具调用和下游系统，沙盒边界就必须被明确设计、测试和审计。没有边界，模型会从问答组件变成攻击路径中的转接站。

6 中低与新兴风险：代理、RAG、错误信息与成本失控

6.1 中低风险怎样升级

S6 承接 OWASP LLM Top 10 2025 的后五项：LLM06:2025 Excessive Agency、LLM07:2025 System Prompt Leakage、LLM08:2025 Vector and Embedding Weaknesses、LLM09:2025 Misinformation、LLM10:2025 Unbounded Consumption。以下不重新定义官方条目，只讨论后五项在权限、RAG、信息可信度和资源预算中的升级条件。讲者把它们称为中等和新兴风险，并不表示可以轻视。它们常常从信息层、配置层或成本层起步，一旦叠加自动化、跨会话影响、工具调用和共享资源，就会抬升为系统级事故。

中低风险的升级条件主要有四个：权限足够大，自动化足够快，影响能跨会话传播，成本或业务后果由组织承担。满足这些条件后，系统提示泄露、错误信息和资源消耗都会从“局部问题”变成真实安全问题。

可以用一个分层视角理解后五类风险：LLM06 关心 Agent 能做什么，LLM07 关心攻击者能知道什么，LLM08 关心 RAG 能取到什么，LLM09 关心错误内容会被谁相信，LLM10 关心资源会被消耗到什么程度。五者经常互相放大：系统提示泄露帮助构造更准的提示注入，过度代理把错误输出变成真实动作，向量库污染制造错误信息，无界消耗拖垮整个服务。

⁷ 视频画面时间区间：00:15:50–00:15:51。

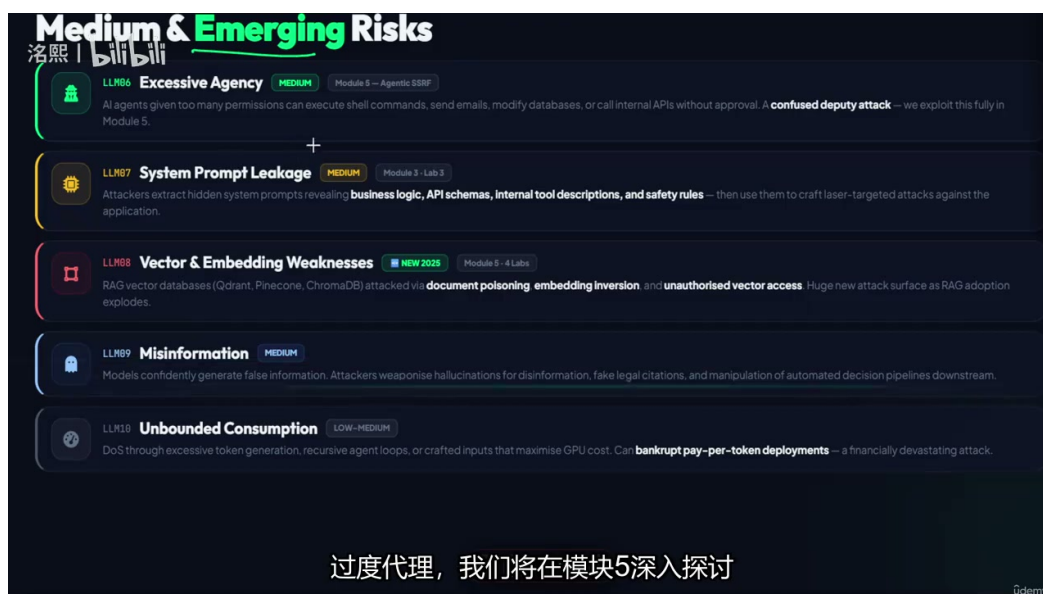


图 11: S6 中低与新兴风险列表的清晰帧：过度代理、系统提示泄露、向量与嵌入弱点、错误信息、无界消耗被放在同一页，便于承接 LLM06 到 LLM10 的分层讲解。⁷

6.2 LLM06:2025 Excessive Agency（过度代理）：能力越大，失控面越宽

LLM06:2025 Excessive Agency 的关键在“模型或 Agent 获得了超过任务所需的能力”。讲者列举了执行 shell 命令、发送邮件、修改数据库、调用内部 API 等动作。只要模型能直接触发这些动作，输入层面的诱导就可能变成真实世界的写入、发送、删除、转账、审批或部署。

Agent 系统通常由三部分组成：语言模型负责理解和规划，工具层负责执行动作，编排器负责循环、记忆和状态管理。权限过大时，三者会形成危险组合：模型判断可能出错，工具执行没有足够校验，编排器还会不断重试或分解任务。混淆代理（confused deputy）就在这里出现：一个拥有权限的代理被低权限用户或外部内容诱导，替攻击者使用自己的合法权限。

Agent 权限不能按“功能方便”授予，应按任务最小闭包授予。能读就不要给写，能草稿就不要自动发送，能查询就不要直接修改，能在沙盒执行就不要接入生产凭据。权限边界越粗，提示注入后的爆炸半径越大。

Agent 风险可以用状态转移来表示：

$$s_{t+1} = f(s_t, m(x_t, c_t), a_t)$$

其中， s_t 是当前系统状态， x_t 是用户输入或外部内容， c_t 是上下文和记忆， $m(\cdot)$ 是模型生成的决策， a_t 是工具动作， s_{t+1} 是执行后的新状态。若 a_t 包含写数据库、发邮件或执行 shell，模型的一次误判会直接改变系统状态。安全设计必须限制 a_t 的范围，并对高风险状态转移设置确认、审批和回滚。

6.3 LLM07:2025 System Prompt Leakage（系统提示泄露）：攻击者拿到操作说明书

LLM07:2025 System Prompt Leakage 在本节聚焦隐藏系统提示、业务逻辑、API 架构、内部工具描述、安全规则和权威指令被诱导披露后的攻击价值。讲者强调，攻击者拿到系统提示的精确语言后，可以对应用发起更精准的攻击。原因很简单：系统提示常常写着模型应该遵循的层级、拒绝规则、工具名称、参数格式、内部角色和异常处理方式。

系统提示可以类比成一份操作说明书。说明书泄露后，攻击者知道哪些按钮存在、哪些门禁会检查、哪些术语会触发信任、哪些规则可以被绕开。即便系统提示泄露本身没有直接给出数据库密码，它也会显著降低后续提示注入、越权调用、数据枚举和过滤绕过的成本。

系统提示泄露和敏感信息泄露有重叠，但侧重点不同。敏感信息泄露关注秘密内容本身；系统提示泄露关注“系统怎样工作”。后者常常是攻击准备阶段的情报收集，价值在于帮助攻击者构造更贴合内部规则的输入。

防护不能依赖“提示词保密”这一层。系统提示应避免包含真实密钥、内部 token 和不可公开的高价值秘密；工具权限应由后端鉴权决定；安全策略应在服务端执行；模型拒绝规则应被外部策略引擎、结构化校验和审计系统补强。把系统提示当成机密可以提高门槛，把系统提示当成唯一控制面会制造脆弱性。

6.4 LLM08:2025 Vector and Embedding Weaknesses（向量与嵌入弱点）：RAG 把知识库变成攻击面

LLM08:2025 Vector and Embedding Weaknesses 是 2025 版中特别贴近 LLM 应用工程的一类。RAG 会把文档切块、嵌入、写入向量数据库，再根据用户查询召回相关片段交给模型。Qdrant、Pinecone、ChromaDB 等向量数据库承担了存储和相似度检索角色。这个链路一旦进入生产，攻击面就从聊天框扩展到文档摄取、权限过滤、嵌入版本、索引更新和召回排序。

向量数据库可以理解为“语义索引仓库”。传统数据库按字段精确查询，向量库按语义距离召回相近内容。若把一段文本嵌入为向量 $\mathbf{q}$ ，文档片段嵌入为 $\mathbf{d}_i$ ，常见相似度可以写成：

$$\cos(\mathbf{q}, \mathbf{d}_i) = \frac{\mathbf{q} \cdot \mathbf{d}_i}{\|\mathbf{q}\| \|\mathbf{d}_i\|}$$

其中， $\mathbf{q}$ 是查询向量， $\mathbf{d}_i$ 是第 i 个文档向量，点积 $\mathbf{q} \cdot \mathbf{d}_i$ 衡量方向接近程度， $\|\cdot\|$ 表示向量长度。安全含义在于：检索结果由嵌入模型、切块策略、权限过滤和相似度排序共同决定，任何一处失控都可能把错误或未授权内容送进模型上下文。

典型问题包括：未授权用户召回不该看的向量数据；知识库被污染，恶意文档在相关查询中稳定排前；嵌入模型升级后新旧向量混用，导致召回漂移；不同租户的数据进入同一索引，权限过滤滞后；外部网页或 PDF 中的间接提示注入被 RAG 当成可信上下文。RAG 的安全边界必须覆盖摄取、索引、检索和生成全流程。

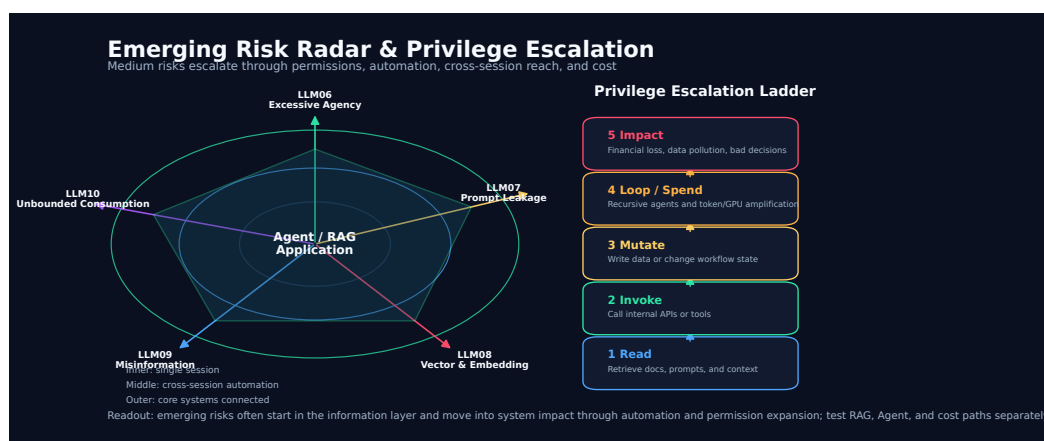


图 12: S6 新兴风险雷达与权限升级图：中低风险常从信息层起步，在自动化、跨会话、工具调用和成本放大条件下抬升为系统级影响。

6.5 LLM09:2025 Misinformation（错误信息）：幻觉被自动化放大

LLM09:2025 Misinformation 关注模型生成虚假、不准确或误导性信息，并造成错误传播或错误决策。讲者提到武器化幻觉、虚假信息传播、伪造法律引用和操控下游自动化决策。这里的风险已经超出“模型偶尔说错”。当错误内容被包装成权威答案、被批量传播、被写入数据库、被用于审批或被下游 Agent 当作事实执行，它就成为攻击者可利用的材料。

幻觉武器化有三条常见路径。第一，伪造来源，例如编造论文、判例、政策条款或安全公告，让用户相信不存在的依据。第二，定向宣传，把模型生成能力用于规模化、个性化的误导内容。第三，污染自动化决策，让错误事实进入风控、客服、合规、招聘、医疗或法律流程，形成后续连锁判断。

错误信息在低信任闲聊中主要是质量问题，在高信任自动化流程中就是安全问题。越是接近法律、医疗、金融、身份审核、舆情传播和生产运维，越需要来源校验、置信度标注、人类复核和可追溯证据。

实践清单很朴素：要求回答带可验证来源；区分模型生成内容和检索证据；对法律、医疗、金融等高风险内容设置人工复核；禁止把无来源结论直接写入关键数据库；对批量生成和传播行为设置速率限制与审计。

6.6 LLM10:2025 Unbounded Consumption（无界消耗）：token、GPU 与递归循环

LLM10:2025 Unbounded Consumption 在本节聚焦 token、API 调用、GPU、存储、带宽、工具执行次数或 Agent 循环失控怎样演变为可用性和成本事故。讲者提到过度 token 生成、DoS、递归代理循环、最大化 GPU 成本、耗尽免费额度和造成财务损失。这类风险看似偏运营成本，生产环境里会直接影响可用性和预算。

成本失控可以用一个粗略模型表达：

$$\text{Cost} = \sum_{i=1}^N \left(p_{\text{in}} T_{\text{in}}^{(i)} + p_{\text{out}} T_{\text{out}}^{(i)} + p_{\text{tool}} K_i + p_{\text{gpu}} G_i \right)$$

其中， N 是请求次数， $T_{\text{in}}^{(i)}$ 和 $T_{\text{out}}^{(i)}$ 分别是第 i 次请求的输入和输出 token 数， K_i 是工具调用次数， G_i 是 GPU 推理资源量， p_{in} 、 p_{out} 、 p_{tool} 、 p_{gpu} 分别表示对应单价。攻击者会尝试增大请求次数、上下文长度、输出长度、工具调用次数和递归深度，让组织为无效工作付费。

递归 Agent 循环尤其危险。一个 Agent 若把自己的输出作为下一轮输入，又缺少停止条件、预算上限和任务去重，就会陷入“计划、调用、观察、再计划”的循环。单个用户会话消耗过大已经麻烦；若攻击可以批量触发，或消耗共享 GPU 队列和组织级额度，就会演变成 DoS 与财务攻击。

无界消耗的防护核心是预算化执行：每个用户、租户、会话、任务和工具都应有 token 上限、时间上限、递归深度上限、并发上限、成本上限和异常熔断。没有预算边界，自动化能力越强，亏损和不可用越快到来。

6.7 MITRE ATLAS：下一讲入口

视频尾部从无界消耗转向 MITRE ATLAS。字幕把它识别成 MEI 或 meatlas，需要统一校正为 MITRE ATLAS，即 Adversarial Threat Landscape for Artificial-Intelligence Systems。讲者的实质收束是：先掌握 OWASP LLM Top 10 这些概念，下一讲再进入 MITRE ATLAS 这一 AI 对抗威胁知识库和战术框架。结尾的无关平台话术和寒暄不属于正文知识点，应过滤。

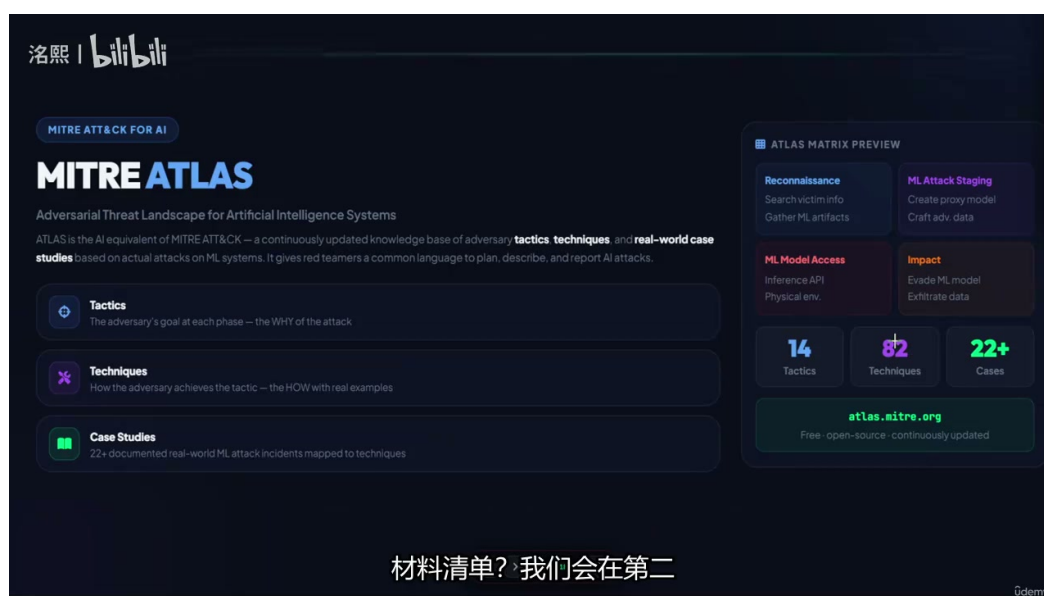


图 13: S6 结尾的 MITRE ATLAS 引子帧：画面给出 tactics、techniques、case studies 和 ATLAS matrix preview，为下一讲的 AI 对抗威胁知识库铺垫入口。⁸

⁸视频画面时间区间：00:18:18–00:18:19。

MITRE ATLAS 的价值在于把 AI 攻击从“风险清单”推进到“战术、技术和案例库”。OWASP LLM Top 10 适合回答“风险类别是什么”，MITRE ATLAS 更适合回答“攻击者会按什么阶段和技术路径行动”。本节只把它作为下一讲入口，不展开矩阵细节。

6.8 本章小结

本章深化了 LLM06 到 LLM10。过度代理关注 Agent 权限过大，系统提示泄露关注内部规则和工具说明暴露，向量与嵌入弱点关注 RAG、Qdrant、Pinecone、ChromaDB 和向量数据权限，错误信息关注幻觉被武器化后的传播和决策影响，无界消耗关注 token、GPU、递归 Agent 循环和 DoS 成本攻击。

这些风险的共同判断标准是升级条件：是否有权限，是否自动执行，是否跨会话传播，是否消耗共享资源，是否影响高信任决策。满足这些条件后，中低风险会迅速变成生产事故。最后，MITRE ATLAS 将作为下一讲入口，负责把 OWASP 的分类地图接到后续对抗战术与技术矩阵。

6.9 全讲总结与延伸

6.9.1 讲者的实质收束

讲者在最后没有继续扩展新技术细节。他的收束动作很明确：第一，掌握 OWASP LLM Top 10 2025 的十类概念；第二，下一讲进入 MITRE ATLAS。结尾的无关平台话术没有教学价值，正文应当丢弃。真正有价值的收束是学习路径：先用 OWASP 建立风险分类，再用 MITRE ATLAS 进入攻击者战术和技术库。

6.9.2 核心主张

本讲的核心主张可以压缩为三句。第一，AI 红队关注行为漏洞，不能照搬传统渗透测试的确定性思维。第二，LLM 应用的风险来自完整工程链路，覆盖数据、模型、供应链、提示、RAG、Agent、输出处理和资源预算。第三，风险等级取决于系统连接了什么；连接越多，权限越大，自动化越强，模型错误或诱导越容易变成真实损害。

6.9.3 机制压缩

从机制上看，十类风险可以归入四条主线。控制线包括提示注入、系统提示泄露和过度代理，核心是攻击者争夺指令、情报和执行能力。数据线包括敏感信息泄露、数据与模型投毒、向量与嵌入弱点，核心是秘密、训练材料和检索知识库的可信边界。集成线包括供应链和输出处理不当，核心是外部组件与下游系统如何解释模型产物。运营线包括错误信息和无界消耗，核心是错误内容与资源成本如何被规模化放大。

最短的安全记忆法：输入要分级，数据要隔离，组件要验真，输出要校验，Agent 要限权，RAG 要控权，成本要封顶，证据要可追溯。做不到这些，LLM 应用就会把语言不确定性传导到工程系统里。

6.9.4 实践清单

落地时可以按八项检查推进：一，列出模型能读到的所有上下文；二，列出模型能调用的所有工具；三，确认敏感信息是否进入提示、日志或 RAG；四，校验模型、LoRA、pip 包、数据集和序列化对象来源；五，为下游输出加 schema、编码、净化和权限检查；六，为 Agent 设置最小权限、人工确认、审计和回滚；七，为向量库加租户隔离、权限过滤、嵌入版本管理和污染检测；八，为 token、GPU、工具调用、递归深度和并发设置预算上限。

6.9.5 开放问题

仍然需要继续追问的问题有四个。第一，组织怎样衡量提示注入测试的覆盖率，避免只测几个模板化越狱句子。第二，RAG 系统如何在语义召回和权限隔离之间取得可验证的平衡。第三，Agent 的自主性、效率和安全审批之间怎样设计可接受的工作流。第四，错误信息的责任边界如何落到产品、模型、检索来源和最终使用者之间。

6.9.6 全讲收束

S5 和 S6 把 OWASP LLM Top 10 从名称表推进到攻击机制。前五项更像高危链路，容易直接走向泄露、投毒、RCE、SSRF 和攻击中继；后五项更像新兴放大器，会通过权限、RAG、幻觉和资源消耗把局部问题推向系统级影响。下一讲进入 MITRE ATLAS 后，读者应带着同一个问题继续看：攻击者按什么战术路径行动，防守者在哪些边界切断链路。

6.9.7 交付件与复核入口

后续复核应直接回到原始字幕和图像清单，而非只看 PDF 成品：assets/OWASP LLM Top 10 深度解读.ai-zh.srt 保留字幕，sections/outline_contract.json 记录章节边界与术语所有权，sections/figure_manifest_*.json 记录图像来源，logs/consistency_report.md 记录一致性检查。复核判准很简单：字幕中的具体工具、阶段、权限动作、数据类型、产品名或成本后果，应在对应章节有解释；正文引用视频帧时，图注应说明画面内容并给出明确时间区间。若下一轮复核发现遗漏，应优先修正文稿和图注，再重新编译 PDF 与运行布局检查。